Automatic ST Code Generation for CODESYS

GenAI Course Project

Shen Pengyu Zhang Yabin Xi Bowen

Zhejiang University

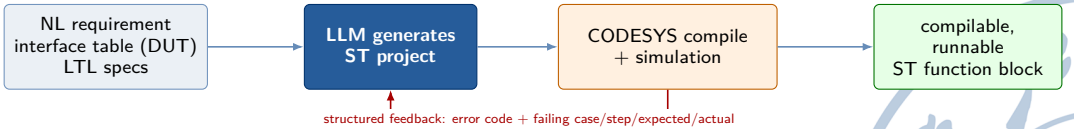
November 11, 2026

Outline

1. Problem Overview
2. Data and Specifications
3. Agent Framework
4. Results and Explanation
5. Conclusion



Problem Overview



Not about "looking right"

- Syntax and project structure: DUTs, Standard library, call graph
- Behaviour: state machines, timers, edges, priority
- Trustworthy judging needs **real execution**

What counts as solved

Compiles **and** passes every test step in simulation;
cases hidden from the model

Deliverables

30 tasks with specs (75 LTL / 54 cases)
Self-built harness · CODESYS compile + simulate
One shared evaluation pipeline

Four settings

{Codex CLI, self-built} × {DeepSeek, Qwen}
30 tasks each, same 5-round budget, no tool errors

Dataset

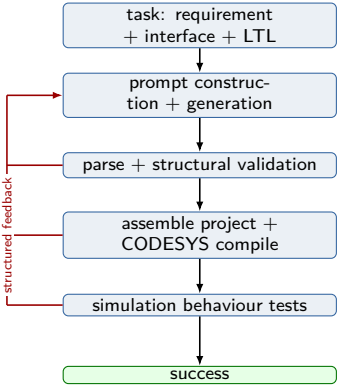
Dimension	Count
Tasks	30 (20 public + 10 self-built)
Difficulty	easy 9 / medium 12 / hard 9
Formal specs	75 (2.5 per task)
Test cases	54 groups, 199 scan steps

Each task: `FUNCTION_BLOCK` interface (incl. DUT) + ZH/EN requirement + LTL + behavioural cases + compilable reference

- **Self-built**: traffic light, water tank, automatic door, sorting, duty/standby pump, alarm latch, star-delta, parking, constant-pressure supply, elevator

- **Diversity**: discrete manufacturing / process industry / motors / safety / queues / strings; 3 tasks with DUTs, 4 with `VAR_IN_OUT`
- **Specs**: mutual exclusion, response, persistence, range invariants, error codes
- **Cases**: normal flow, boundaries, priority conflicts, error handling; **every step states its purpose** and links to an LTL spec
- **Self-check**: reference implementations **30/30** compile and **30/30** pass their own cases

Architecture: Single-Agent Feedback Loop



- One full project per round, rewritten wholesale, never patched
- **Validation up front**: format/interface errors cost no CODESYS cold start
- **Context**: keep only the last 2 rounds; errors deduplicated by (code, text)
- **Stop**: pass / 5 rounds / code unchanged / repeated failure signature / tool error
- Test cases **hidden from the model**; simulation runs on the official CODESYS runtime

Experimental Setup

	deepseek-v4.1-flash-1017	qwen3.7-flash
Existing framework (Codex CLI)	Setup 1	Setup 2
Self-built harness	Setup 3	Setup 4

- Codex CLI is wrapped as the same `chat(messages)` client as our harness: **identical prompt, feedback text, round limit, compile/test pipeline**
- The only difference is how generation is organised (Codex brings its own agent loop and tool context), so differences attribute to the framework itself
- **Metrics**: first-round compile, final compile, test pass rate, tokens, wall time, rounds

Overall: Tie on DeepSeek, Gap on Qwen

Setting	First compile	Final compile	Tests pass	Avg. rounds	Avg. time
Self-built + DeepSeek	96.7%	100.0%	**83.3%**	1.77	163 s
Self-built + Qwen	76.7%	93.3%	**66.7%**	2.53	295 s
Codex + DeepSeek	100.0%	100.0%	**83.3%**	1.70	342 s
Codex + Qwen	30.0%	83.3%	**46.7%**	3.53	315 s

83.3% / 83.3%

DeepSeek: both frameworks **tie exactly**

66.7% vs 46.7%

Qwen: self-built is **20 points ahead**

96.7% vs 100%

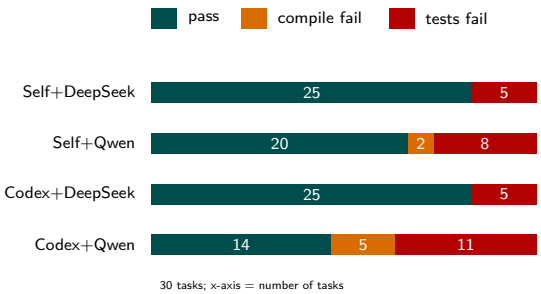
first round: feedback closes the gap

Cost: Same Quality, an Order of Magnitude Less Input

Setting	Input tokens	Output tokens	Avg. time	Relative to Codex
Self-built + DeepSeek	75,058	383,312	163 s	input **13%**, time **48%**
Self-built + Qwen	131,255	503,606	295 s	input **9%**, time **94%**
Codex + DeepSeek	565,110	521,147	342 s	—
Codex + Qwen	1,404,958	394,781	315 s	—

- At the same pass rate (83.3%), the harness uses **1/7** of the input tokens and about **1/2** the wall time
- Nearly all of the gap is on the **input side**: Codex's agent loop re-accumulates system prompt, tool definitions and history
- On Qwen the harness takes about the same time as Codex (94%) yet passes 20 points more
- Output tokens are comparable (383k vs 521k): both frameworks "think" a similar amount, they differ in how much they "read"

Why 1: The Gap Sits at Compile, Not Behaviour



- On DeepSeek both frameworks have **zero** compile failures: the compile gate is no longer the bottleneck, all losses are behavioural
- Self-built + Qwen leaves only 2 tasks uncompilable, feedback pushes the other 28 into testing
- Codex + Qwen has **5 tasks still failing to compile**: 30% first round, so the 5-round budget burns on syntax
- Among compilable tasks the behavioural pass rates are 83.3% / 83.3% / 71.4% / 56.0%: on Qwen the gap narrows **from 20 points to 15**, the rest comes from tasks that never compiled

Why 2: Where the Qwen Gap Comes From

Metric (Qwen)	Self-built	Codex
Reach compiler, 1st round	**30/30**	**21/30**
blocked, interface check	0	7
blocked, block parsing	0	2
Compile errors per round	**1.17**	**2.66**
1st-round errors (med./mean)	0 / 1.2	2 / 3.7
Compiles on 1st round	23/30	9/30
Avg. rounds, tasks both pass	**1.38**	**2.00**

The failing sets largely overlap — Codex needs **more rounds to converge at the same capability** (limit: 5).

- **Output structure does not hold**: Codex emits `REAL/BOOL` as DUT names, array aliases as `kind=array`, or no POU block at all — 9 submissions never reached the compiler
- **Errors look C-like**: `C0189` 74, `C0968` samples read `expected ',' instead of ')'` or `instead of '!'` — ST written as C/JS (`!`, `if(...)`, `//`)
- **Prompt layer dilution**: Codex runs as a general agent, so the model must satisfy both "I am a coding agent" and "emit one FB in a strict block format"; Qwen follows instructions weakly and slides to generic style — why DeepSeek ties and Qwen diverges
- **Context layer**: median input 11,314 vs 1,573 tokens (7.2×), squeezing the output budget (3,354 vs 6,101)

Conclusion

1. A 30-task dataset with per-task LTL specs, test cases (each step with a stated purpose) and reference implementations; all 30/30 compile and pass simulation
2. The harness matches Codex on DeepSeek (83.3% tests pass) at roughly 1/7 of the input tokens and 1/2 of the wall time; on Qwen it is still better (66.7% vs 46.7%)
3. The gap lies mainly in **the share that reaches the compiler** (30/30 vs 21/30), error density per round (1.17 vs 2.66) and context cost — not in "the model thinking too little"

Limitations

- Specs are not hooked to PLCVerif (optional item, not done); one sample per task, and the Qwen difference has Fisher exact $p = 0.19$, not significant at the 0.05 level
- 12 scan steps carry no assertion and 34 BOOL inputs are never set TRUE, so pass rates hold "under the current test cases"
- Next: PLCVerif integration, stronger case coverage, other platforms

Turning "does it compile and does it run correctly" into structured feedback lifts weak models; but no framework is better for every model.